SkillCraft1 Master Table Dataset

Multinomial Skill Classification Using Machine Learning

William Chen

University of California, Irvine

DATA 200BP: Intermediate Probability and Statistical Theory II

Abstract

This project extends the work of Thompson et al. (2013), who used pairwise binary classification with Conditional Inference Forest to study variable importance across StarCraft II skill levels. While their approach provided insights into how predictors of expertise change across leagues, it did not address the full complexity of multi-class skill prediction or the class imbalance inherent in the SkillCraft1 dataset. This study reformulates the problem as a 6-class multinomial classification task, predicting player league rankings from Bronze (1) to Masters (6).

Three machine learning models are evaluated — Multinomial Logistic Regression, Decision Tree, and Random Forest — each trained under three class imbalance strategies: Baseline, Class Weights, and SMOTE combined with Tomek Links. Model performance is assessed using overall accuracy and minority class recall on a held-out test set.

Results show that Random Forest with Class Weights achieves the best overall balance, with an accuracy of 0.4070 and a meaningful improvement in minority class recall compared to the baseline. In a direct pairwise comparison replicating Thompson et al.'s setup, our

best model outperforms their Conditional Inference Forest results in 3 out of 4 league pairs. These findings suggest that class weighting is an effective and practical strategy for handling imbalance in multi-class skill classification tasks.

1. Introduction

1.1 Nature of the Project

Cognitive science has long studied expertise by comparing experts with novices. Thompson et al. (2013) challenged the assumptions underlying such comparisons by analyzing StarCraft II telemetry data from 3,395 players across competitive leagues. They discovered that variable importance is not static — APM dominates at lower skill levels, but at higher levels, attentional efficiency metrics become more predictive.

Their method used pairwise binary classifiers built with Conditional Inference Forests, each comparing players two leagues apart. This design was deliberate: neighboring leagues are not cleanly separable because league placement does not perfectly reflect objective skill. They found that classifiers performed poorly on adjacent leagues and had significantly more success when the distance between classes was at least two.

StarCraft II is well-suited for this type of study. Unlike chess, an average RTS game contains ~1,635 moves, providing dense behavioral data in a naturalistic setting. The SkillCraft1 dataset contains 20 behavioral and demographic variables, making it the largest expertise study of its kind.

1.2 Aims

This project extends Thompson et al.'s pairwise approach in two directions. First, we replicate their pairwise binary classifiers using our best-performing configuration to benchmark our methodology against their Conditional Inference Forest results. Second, we develop a direct 6-class multinomial prediction model that simultaneously classifies players across all skill levels — a more challenging problem that requires handling severe class imbalance and non-linear decision boundaries.

To address class imbalance, three strategies are compared: baseline classification, class weight adjustment, and SMOTE combined with Tomek Links. This allows us to investigate whether imbalance-aware training improves prediction of minority classes.

1.3 Goals

The primary goal of this project is to develop an effective 6-class multinomial classifier for Leagues 1 through 6 that balances overall accuracy with minority class recall. To achieve this, we compare three machine learning models — Multinomial Logistic Regression, Decision Tree, and Random Forest — each trained under three imbalance strategies: Baseline, Class Weights, and SMOTE combined with Tomek Links. Additionally, we replicate Thompson et al.'s pairwise binary classification setup to benchmark our best-performing model against their Conditional Inference Forest results.

A secondary goal is to examine whether demographic features such as Age and HoursPerWeek contribute meaningfully to skill prediction beyond in-game behavioral metrics.

2. Literature Review

2.1 Skill Classification in Competitive Gaming

Skill assessment in competitive games bridges expertise research with machine learning. Unlike laboratory settings, games provide naturalistic data where players compete under identical conditions. Real-time strategy games like StarCraft II are particularly rich: they demand simultaneous management of multiple objectives and high-frequency decision-making, making them ideal for studying expertise across a continuous skill spectrum.

2.2 Thompson et al. (2013): SkillCraft1 Baseline

Thompson et al. (2013) introduced the SkillCraft1 dataset, comprising 3,395 players across 7 leagues with 20 behavioral and demographic variables. Their primary objective was to examine how variable importance changes across skill levels, rather than to maximize classification accuracy. Using pairwise binary classifiers built with Conditional Inference Forests (ntree = 1000, mtry = 5), they compared leagues two levels apart to avoid the separability issues inherent in adjacent league comparisons.

Their analysis revealed that adjacent leagues are difficult to separate because league placement does not perfectly reflect objective skill. They also found that variable importance shifts across the skill spectrum — APM predicts skill at lower levels, while attentional

efficiency metrics such as Action Latency and Gap Between PACs become dominant at higher levels. Grandmaster players were excluded due to their small sample size (35 players), and 55 professional players were added from publicly available tournament replays to form the highest skill group.

This project extends their work by replicating the pairwise comparison using RF + Class Weights — a standard Random Forest with class weighting, as opposed to their Conditional Inference Forest — and by attempting direct 6-class multinomial prediction, a harder problem that Thompson et al. did not pursue due to the separability limitations they observed.

2.3 Class Imbalance Handling Methods

Class imbalance occurs when certain classes are significantly smaller than others, causing models to favor majority classes and underperform on minority ones. Three common strategies for addressing this issue are evaluated in this study.

SMOTE generates synthetic minority samples by interpolating between existing observations and their nearest neighbors. This increases representation without duplication, but may produce samples that do not reflect realistic data patterns.

Tomek Links remove majority class samples that are nearest neighbors of minority class samples, cleaning the decision boundary. This is often combined with SMOTE to provide both augmentation and boundary clarity.

Class Weighting adjusts the loss function to penalize misclassification of minority samples more heavily, typically using inverse frequency weighting. Unlike resampling methods, it preserves the original data distribution entirely.

2.4 Machine Learning Models for Classification

Multinomial Logistic Regression produces linear decision boundaries and interpretable coefficients, but may underperform when feature-class relationships are nonlinear.

Decision Trees partition the feature space using binary splits, naturally handling nonlinearity. However, they are prone to overfitting and require pruning to generalize.

Random Forests aggregate many decision trees through bagging and random feature selection, reducing overfitting and improving robustness at the cost of interpretability.

These three models are compared across three imbalance strategies to identify the most effective combination for 6-class skill classification.

3. Data Introduction

3.1 SkillCraft1 Dataset Overview

The SkillCraft1 dataset contains 3,395 StarCraft II player observations across 8 leagues. After removing League 8 (systematic missing data in all demographic features) and League 7 (Grandmaster, excluded per Thompson et al. due to small sample size), the analysis uses 3,340 observations across 6 leagues: Bronze, Silver, Gold, Platinum, Diamond, and Masters.

The dataset is severely class-imbalanced, with Bronze players comprising only about 5% of observations while Platinum and Diamond together account for nearly half.

3.2 Data Features & Characteristics

The dataset contains 20 variables: 3 demographic (Age, HoursPerWeek, TotalHours) and 17 behavioral, capturing in-game actions such as APM, hotkey usage, map control, decision-making efficiency, and unit production, all normalized per minute of gameplay.

4. Exploratory Data Analysis (EDA)

4.1 Data Loading and Structure

In [2]:
```
library(ggplot2)
library(rpart)
library(rpart.plot)
library(cowplot)
library(caret)
library(randomForest)
```

```r
library(tidyr)
library(GGally)
library(corrplot)
library(pROC)
library(nnet)
library(car)
library(dplyr)
library(smotefamily)
library(themis)
library(nnet)
options(warn = -1)
```

In [3]:
```r
URL = "https://archive.ics.uci.edu/ml/machine-learning-databases/00272/SkillCraft1_Dataset.csv"
skillcraft.data <- read.csv(URL)
```

In [4]:
```r
str(skillcraft.data)
```

```
'data.frame':   3395 obs. of  20 variables:
 $ GameID              : int  52 55 56 57 58 60 61 72 77 81 ...
 $ LeagueIndex         : int  5 5 4 3 3 2 1 7 4 4 ...
 $ Age                 : chr  "27" "23" "30" "19" ...
 $ HoursPerWeek        : chr  "10" "10" "10" "20" ...
 $ TotalHours          : chr  "3000" "5000" "200" "400" ...
 $ APM                 : num  144 129 70 108 123 ...
 $ SelectByHotkeys     : num  0.00352 0.0033 0.0011 0.00103 0.00114 ...
 $ AssignToHotkeys     : num  0.00022 0.000259 0.000336 0.000213 0.000327 ...
 $ UniqueHotkeys       : int  7 4 4 1 2 2 6 6 2 8 ...
 $ MinimapAttacks      : num  1.10e-04 2.94e-04 2.94e-04 5.33e-05 0.00 ...
 $ MinimapRightClicks  : num  0.000392 0.000432 0.000461 0.000543 0.001329 ...
 $ NumberOfPACs        : num  0.00485 0.00431 0.00293 0.00378 0.00237 ...
 $ GapBetweenPACs      : num  32.7 32.9 44.6 29.2 22.7 ...
 $ ActionLatency       : num  40.9 42.3 75.4 53.7 62.1 ...
 $ ActionsInPAC        : num  4.75 4.84 4.04 4.92 9.37 ...
 $ TotalMapExplored    : int  28 22 22 19 15 16 15 45 29 27 ...
 $ WorkersMade         : num  0.001397 0.001193 0.000745 0.000426 0.001174 ...
 $ UniqueUnitsMade     : int  6 5 6 7 4 6 5 9 7 6 ...
 $ ComplexUnitsMade    : num  0 0 0 0 0 ...
 $ ComplexAbilitiesUsed: num  0.00 2.08e-04 1.89e-04 3.84e-04 1.93e-05 ...
```

The dataset contains 3,395 observations and 20 variables. Based on the structure, the following conversions are required:

- LeagueIndex: ranges from 1 (Bronze) to 8 (Professional), converted to factor for classification
- GameID: serves only as an identifier with no predictive value, removed
- Age, HoursPerWeek, TotalHours: stored as character type, converted to numeric

4.2 Data Type Conversion

In [5]:
```
skillcraft.data$LeagueIndex <- as.factor(skillcraft.data$LeagueIndex)
skillcraft.data$GameID <- NULL
skillcraft.data$Age <- as.numeric(skillcraft.data$Age)
skillcraft.data$HoursPerWeek <- as.numeric(skillcraft.data$HoursPerWeek)
skillcraft.data$TotalHours <- as.numeric(skillcraft.data$TotalHours)
```

LeagueIndex is converted to factor for classification. GameID is removed as it has no predictive value. Age, HoursPerWeek, and TotalHours are converted from character to numeric, which introduces NA values for non-numeric entries such as "?".

4.3 Summary Statistics

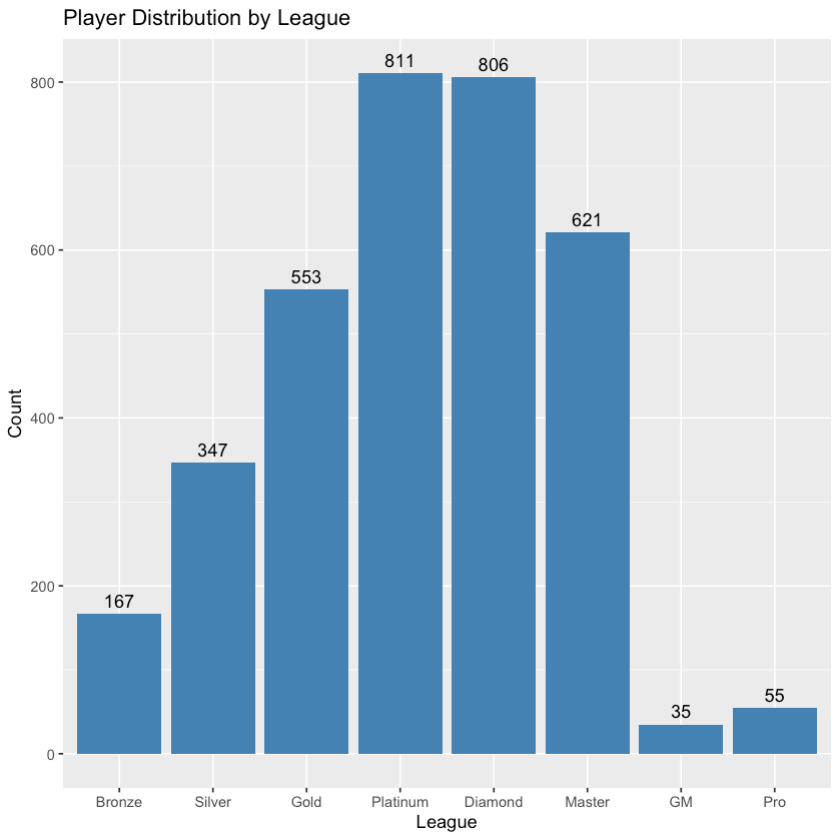
In [6]:
```
summary(skillcraft.data)
```

```
 LeagueIndex       Age         HoursPerWeek      TotalHours
4      :811   Min.   :16.00   Min.   :  0.00   Min.   :      3.0
5      :806   1st Qu.:19.00   1st Qu.:  8.00   1st Qu.:    300.0
6      :621   Median :21.00   Median : 12.00   Median :    500.0
3      :553   Mean   :21.65   Mean   : 15.91   Mean   :    960.4
2      :347   3rd Qu.:24.00   3rd Qu.: 20.00   3rd Qu.:    800.0
1      :167   Max.   :44.00   Max.   :168.00   Max.   :1000000.0
(Other): 90   NA's   :55      NA's   :56       NA's   :57
     APM         SelectByHotkeys    AssignToHotkeys     UniqueHotkeys
Min.   : 22.06   Min.   :0.000000   Min.   :0.0000000   Min.   : 0.000
1st Qu.: 79.90   1st Qu.:0.001258   1st Qu.:0.0002042   1st Qu.: 3.000
Median :108.01   Median :0.002500   Median :0.0003526   Median : 4.000
Mean   :117.05   Mean   :0.004299   Mean   :0.0003736   Mean   : 4.365
3rd Qu.:142.79   3rd Qu.:0.005133   3rd Qu.:0.0004988   3rd Qu.: 6.000
Max.   :389.83   Max.   :0.043088   Max.   :0.0017522   Max.   :10.000

MinimapAttacks      MinimapRightClicks   NumberOfPACs      GapBetweenPACs
Min.   :0.000e+00   Min.   :0.0000000   Min.   :0.000679   Min.   :  6.667
1st Qu.:0.000e+00   1st Qu.:0.0001401   1st Qu.:0.002754   1st Qu.: 28.958
Median :3.993e-05   Median :0.0002815   Median :0.003395   Median : 36.724
Mean   :9.831e-05   Mean   :0.0003874   Mean   :0.003463   Mean   : 40.362
3rd Qu.:1.189e-04   3rd Qu.:0.0005141   3rd Qu.:0.004027   3rd Qu.: 48.291
Max.   :3.019e-03   Max.   :0.0040408   Max.   :0.007971   Max.   :237.143

ActionLatency     ActionsInPAC    TotalMapExplored  WorkersMade
Min.   : 24.09   Min.   : 2.039   Min.   : 5.00    Min.   :7.698e-05
1st Qu.: 50.45   1st Qu.: 4.273   1st Qu.:17.00    1st Qu.:6.830e-04
Median : 60.93   Median : 5.096   Median :22.00    Median :9.052e-04
Mean   : 63.74   Mean   : 5.273   Mean   :22.13    Mean   :1.032e-03
3rd Qu.: 73.68   3rd Qu.: 6.034   3rd Qu.:27.00    3rd Qu.:1.259e-03
Max.   :176.37   Max.   :18.558   Max.   :58.00    Max.   :5.149e-03

UniqueUnitsMade  ComplexUnitsMade    ComplexAbilitiesUsed
Min.   : 2.000   Min.   :0.000e+00   Min.   :0.000e+00
1st Qu.: 5.000   1st Qu.:0.000e+00   1st Qu.:0.000e+00
Median : 6.000   Median :0.000e+00   Median :2.028e-05
Mean   : 6.534   Mean   :5.943e-05   Mean   :1.419e-04
3rd Qu.: 8.000   3rd Qu.:8.554e-05   3rd Qu.:1.814e-04
Max.   :13.000   Max.   :9.023e-04   Max.   :3.084e-03
```

From the summary, Age, HoursPerWeek, and TotalHours contain missing values (NA). Additionally, TotalHours shows an abnormally large maximum value of 1,000,000, suggesting the presence of an outlier that will need to be addressed in preprocessing.

4.4 Class Distribution

In [7]:
```
ggplot(data = skillcraft.data, mapping = aes(x = LeagueIndex)) +
  geom_bar(fill = "steelblue") +
  geom_text(stat = "count", aes(label = ..count..), vjust = -0.5) +
  scale_x_discrete(labels = c("1" = "Bronze", "2" = "Silver", "3" = "Gold",
                              "4" = "Platinum", "5" = "Diamond", "6" = "Master",
                              "7" = "GM", "8" = "Pro")) +
  labs(title = "Player Distribution by League",
       x = "League", y = "Count")
```



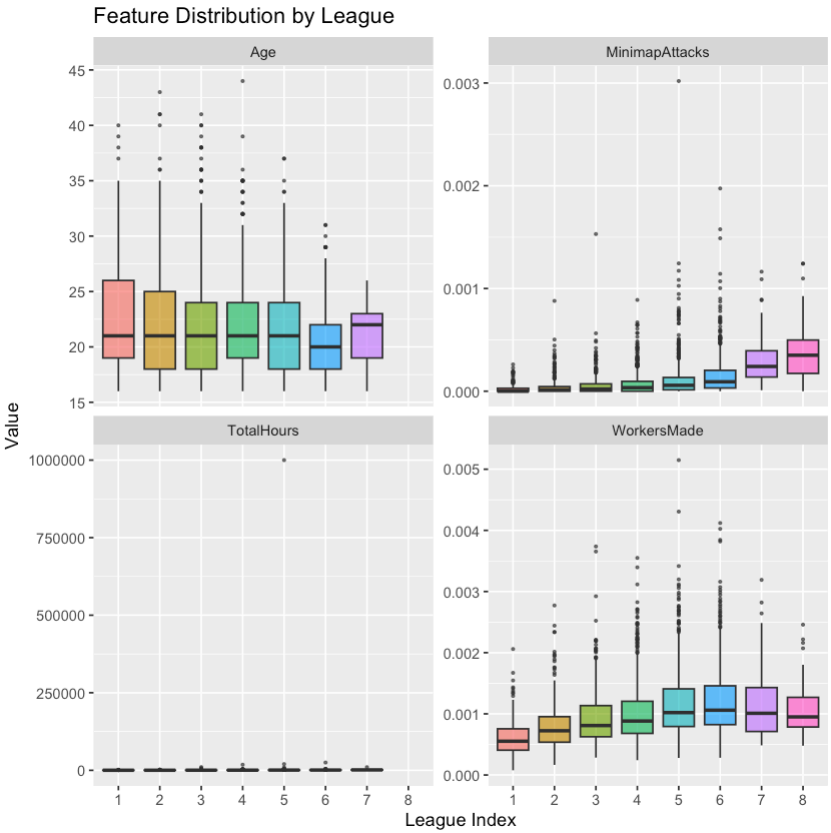
The distribution of players across leagues is highly imbalanced. Platinum (League 4) and Diamond (League 5) account for the majority of observations, while Grandmaster (League 7) and Professional (League 8) have very few samples. This imbalance will be addressed in Section 5 and Section 6.

4.5 Feature Distribution by League

We examine four features that may not intuitively correlate with league rank.

In [8]:
```
features <- c("Age", "TotalHours", "WorkersMade", "MinimapAttacks")

df_long <- pivot_longer(skillcraft.data[, c("LeagueIndex", features)],
                        cols = -LeagueIndex,
```

```
                         names_to = "feature",
                         values_to = "value")

ggplot(df_long, aes(x = LeagueIndex, y = value, fill = LeagueIndex)) +
  geom_boxplot(outlier.size = 0.5, alpha = 0.7) +
  facet_wrap(~ feature, scales = "free_y", ncol = 2) +
  labs(title = "Feature Distribution by League",
       x = "League Index", y = "Value") +
  theme(legend.position = "none")
```



Based on the boxplots:

- Age: Median age is stable around 20–22 across all leagues, suggesting age alone is not a strong indicator of skill level
- MinimapAttacks: Boxes are narrow and close to zero across most leagues, though higher leagues show slightly wider distributions

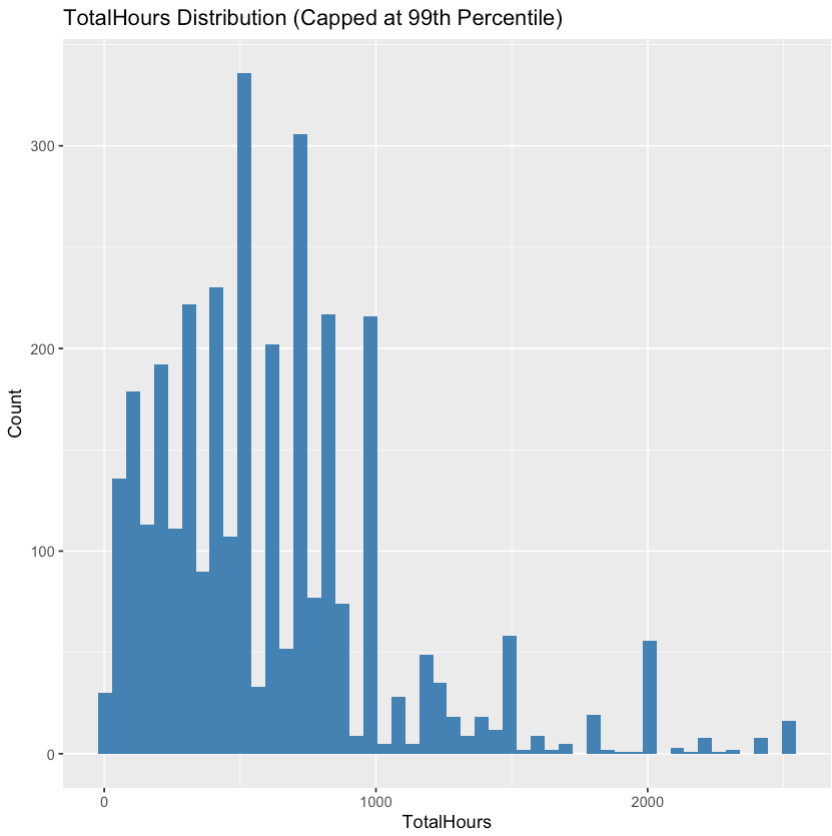
- TotalHours: An extreme outlier (1,000,000 hours) dominates the y-axis, confirming the need for outlier capping in preprocessing
- WorkersMade: League 6 shows the highest median rather than League 7 or 8, suggesting worker production does not linearly increase with rank

4.6 Outlier Analysis (TotalHours)

In [9]:
```
percentiles <- quantile(skillcraft.data$TotalHours,
                        probs = c(0.90, 0.95, 0.99, 0.999, 1.0),
                        na.rm = TRUE)
print(percentiles)
```

```
      90%        95%        99%      99.9%       100%
  1200.00    1566.00    2520.00   15391.62 1000000.00
```

In [10]:
```
cap_99 <- quantile(skillcraft.data$TotalHours, probs = 0.99, na.rm = TRUE)

ggplot(skillcraft.data[skillcraft.data$TotalHours <= cap_99, ],
       aes(x = TotalHours)) +
  geom_histogram(bins = 50, fill = "steelblue") +
  labs(title = "TotalHours Distribution (Capped at 99th Percentile)",
       x = "TotalHours", y = "Count")
```



TotalHours contains an extreme outlier of 1,000,000 hours. After capping at the 99th percentile (2,520 hours), the distribution shows a right-skewed pattern with most players under 1,000 hours. Actual outlier handling is applied in Section 5.

4.7 Missing Value Analysis

We investigate whether missing values are concentrated in specific leagues or spread randomly, as this determines the appropriate treatment.

In [11]:
```
show_missing <- function(league_id) {
  rows <- skillcraft.data[skillcraft.data$LeagueIndex == league_id, ]
  counts <- colSums(is.na(rows))
  counts <- counts[counts > 0]
```

```
  if (length(counts) == 0) return(NULL)
  data.frame(
    League       = paste0("League ", league_id),
    Feature      = names(counts),
    Missing      = as.integer(counts),
    Total        = nrow(rows),
    Missing_Rate = paste0(round(counts / nrow(rows) * 100, 1), "%")
  )
}

result <- as.data.frame(rbind(show_missing(5), show_missing(8)))
print(result, row.names = FALSE)
```

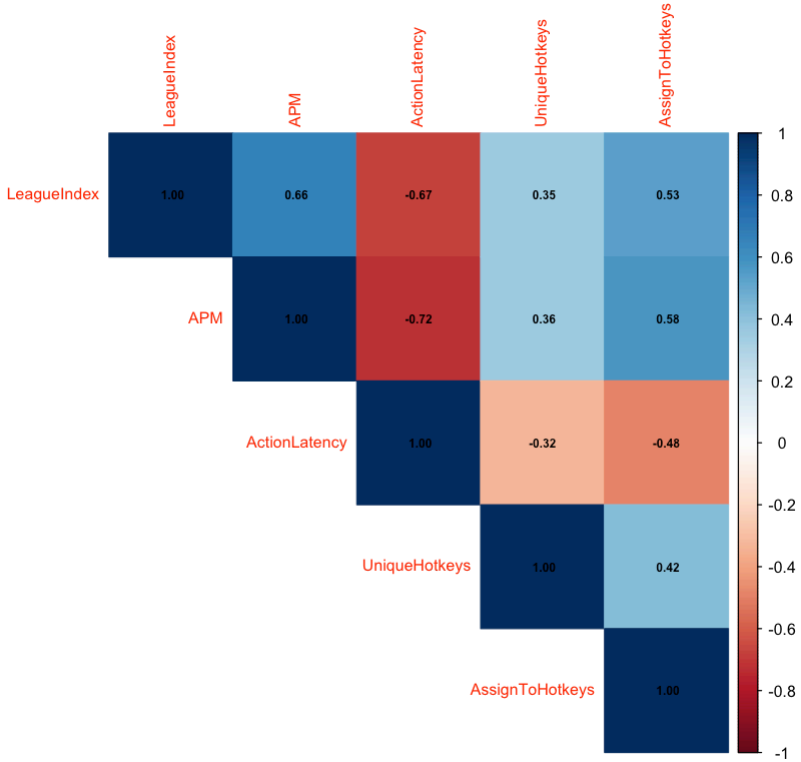
```
  League      Feature Missing Total Missing_Rate
League 5 HoursPerWeek       1   806         0.1%
League 5   TotalHours       2   806         0.2%
League 8          Age      55    55         100%
League 8 HoursPerWeek      55    55         100%
League 8   TotalHours      55    55         100%
```

All 55 League 8 observations are missing Age, HoursPerWeek, and TotalHours (100%), indicating systematic missingness — likely because professional players' data was collected from tournament records rather than standard ladder profiles. In contrast, League 5 has only 2–3 missing entries (< 0.3%), consistent with random gaps. This supports excluding League 8 entirely rather than attempting imputation.

4.8 Feature Correlations

In [12]:

```
numeric_features <- c("LeagueIndex", "APM", "ActionLatency",
                      "UniqueHotkeys", "AssignToHotkeys")

corrplot(cor(skillcraft.data[, numeric_features] %>%
               mutate(LeagueIndex = as.numeric(LeagueIndex)),
             use = "complete.obs"),
         method = "color",
         type = "upper",
         tl.cex = 0.8,
         addCoef.col = "black",
         number.cex = 0.6)
```



The heatmap reveals several notable correlations with LeagueIndex:

- APM (0.66) and AssignToHotkeys (0.53) show strong positive correlations, indicating higher-ranked players have faster actions and more hotkey usage
- ActionLatency (-0.67) shows the strongest negative correlation, confirming that lower reaction time is associated with higher league rank
- APM and ActionLatency are strongly negatively correlated (-0.72), suggesting potential multicollinearity to be addressed in modeling

5. Data Preprocessing

5.1 Exclusion of Leagues 7 & 8

In [13]:
```
skillcraft.data <- skillcraft.data[skillcraft.data$LeagueIndex != 8, ]
skillcraft.data <- skillcraft.data[skillcraft.data$LeagueIndex != 7, ]
skillcraft.data$LeagueIndex <- droplevels(skillcraft.data$LeagueIndex)
```

League 8 (Professional) is excluded due to 100% systematic missing values in all demographic features. League 7 (Grandmaster) is excluded per Thompson et al. (2013) due to its small sample size and analytic difficulties. The remaining dataset contains 3,340 observations across 6 leagues (Bronze through Masters).

5.2 Handle Outliers in TotalHours

In [14]:
```
cap_99 <- quantile(skillcraft.data$TotalHours, probs = 0.99, na.rm = TRUE)
skillcraft.data$TotalHours <- pmin(skillcraft.data$TotalHours, cap_99)
```

TotalHours is capped at the 99th percentile to remove extreme outliers while preserving the overall distribution.

5.3 Handle Remaining Missing Values

In [15]:
```
set.seed(42)
skillcraft.data.imputed <- rfImpute(LeagueIndex ~ ., data = skillcraft.data, iter = 6)
```

```
ntree      OOB      1      2      3      4      5      6
  300:  58.31% 70.66% 68.59% 69.44% 55.36% 57.57% 44.12%
ntree      OOB      1      2      3      4      5      6
  300:  58.49% 67.07% 70.32% 68.17% 55.24% 58.56% 45.09%
ntree      OOB      1      2      3      4      5      6
  300:  58.03% 69.46% 67.44% 70.16% 56.84% 55.46% 43.80%
ntree      OOB      1      2      3      4      5      6
  300:  58.31% 71.26% 69.16% 68.54% 55.73% 58.31% 43.00%
ntree      OOB      1      2      3      4      5      6
  300:  58.25% 67.66% 67.72% 71.79% 54.25% 56.82% 45.41%
ntree      OOB      1      2      3      4      5      6
  300:  58.49% 67.66% 70.89% 71.07% 53.64% 58.93% 43.64%
```

The remaining missing values (5 observations in League 5) are imputed using rfImpute, which uses Random Forest proximity to estimate missing values iteratively over 6 iterations. The OOB error stabilizes around 58–59%, indicating convergence.

5.4 Train/Test Split

In [16]:
```
set.seed(42)
train.index <- sample(1:nrow(skillcraft.data.imputed), 0.8 * nrow(skillcraft.data.imputed))
train.data <- skillcraft.data.imputed[train.index, ]
test.data  <- skillcraft.data.imputed[-train.index, ]
```

The data is split into 80% training (n=2,644) and 20% test (n=661) sets with a fixed random seed.

6. Methods

6.1 Feature Selection & Tuning

Multicollinearity Check (VIF)

In [17]:
```
lm.proxy <- lm(as.numeric(LeagueIndex) ~ ., data = train.data)
vif.result <- vif(lm.proxy)

vif.df <- data.frame(
  Feature = names(vif.result),
  VIF     = round(vif.result, 3)
)
print(vif.df, row.names = FALSE)
```

```
             Feature    VIF
                 Age  1.138
        HoursPerWeek  1.285
          TotalHours  1.380
                 APM 41.052
     SelectByHotkeys 13.756
     AssignToHotkeys  1.555
       UniqueHotkeys  1.292
      MinimapAttacks  1.128
  MinimapRightClicks  1.284
        NumberOfPACs 14.961
      GapBetweenPACs  2.264
       ActionLatency  5.240
        ActionsInPAC  9.417
    TotalMapExplored  1.835
         WorkersMade  1.308
     UniqueUnitsMade  1.659
    ComplexUnitsMade  1.907
ComplexAbilitiesUsed  1.748
```

VIF analysis identifies four features with high multicollinearity: APM (41.05), SelectByHotkeys (13.76), NumberOfPACs (14.96), and ActionsInPAC (9.42). These overlap substantially with other action-based and PAC-related features. ActionLatency (5.24) is also moderately inflated. All remaining features have VIF below 3. High-VIF features will be candidates for removal in the backward elimination step.

Backward Elimination

In [18]:
```
step.result <- stats::step(lm.proxy, direction = "backward")
```

```
Start:  AIC=-255.15
as.numeric(LeagueIndex) ~ Age + HoursPerWeek + TotalHours + APM +
    SelectByHotkeys + AssignToHotkeys + UniqueHotkeys + MinimapAttacks +
    MinimapRightClicks + NumberOfPACs + GapBetweenPACs + ActionLatency +
    ActionsInPAC + TotalMapExplored + WorkersMade + UniqueUnitsMade +
    ComplexUnitsMade + ComplexAbilitiesUsed

                       Df Sum of Sq    RSS      AIC
- Age                   1     0.273 2366.8 -256.842
- MinimapRightClicks    1     0.380 2366.9 -256.722
- ComplexAbilitiesUsed  1     0.887 2367.4 -256.156
- ComplexUnitsMade      1     1.034 2367.6 -255.992
- APM                   1     1.330 2367.8 -255.661
<none>                              2366.5 -255.147
- UniqueUnitsMade       1     2.813 2369.3 -254.006
- HoursPerWeek          1     3.221 2369.7 -253.551
- TotalMapExplored      1     3.756 2370.3 -252.954
- ActionsInPAC          1     4.179 2370.7 -252.482
- SelectByHotkeys       1     8.082 2374.6 -248.132
- UniqueHotkeys         1    11.364 2377.9 -244.481
- NumberOfPACs          1    16.476 2383.0 -238.803
- GapBetweenPACs        1    20.637 2387.2 -234.189
- WorkersMade           1    21.670 2388.2 -233.046
- MinimapAttacks        1    44.589 2411.1 -207.793
- AssignToHotkeys       1    46.586 2413.1 -205.604
- ActionLatency         1    73.393 2439.9 -176.394
- TotalHours            1   197.067 2563.6  -45.661

Step:  AIC=-256.84
as.numeric(LeagueIndex) ~ HoursPerWeek + TotalHours + APM + SelectByHotkeys +
    AssignToHotkeys + UniqueHotkeys + MinimapAttacks + MinimapRightClicks +
    NumberOfPACs + GapBetweenPACs + ActionLatency + ActionsInPAC +
    TotalMapExplored + WorkersMade + UniqueUnitsMade + ComplexUnitsMade +
    ComplexAbilitiesUsed

                       Df Sum of Sq    RSS      AIC
- MinimapRightClicks    1     0.389 2367.2 -258.407
- ComplexAbilitiesUsed  1     0.869 2367.7 -257.871
- ComplexUnitsMade      1     0.993 2367.8 -257.732
- APM                   1     1.404 2368.2 -257.274
<none>                              2366.8 -256.842
```

```
- UniqueUnitsMade       1     2.742 2369.5 -255.780
- TotalMapExplored      1     3.659 2370.4 -254.757
- HoursPerWeek          1     3.687 2370.5 -254.726
- ActionsInPAC          1     4.256 2371.1 -254.092
- SelectByHotkeys       1     8.224 2375.0 -249.670
- UniqueHotkeys         1    11.718 2378.5 -245.783
- NumberOfPACs          1    16.534 2383.3 -240.436
- GapBetweenPACs        1    21.042 2387.8 -235.440
- WorkersMade           1    21.622 2388.4 -234.797
- MinimapAttacks        1    45.277 2412.1 -208.740
- AssignToHotkeys       1    46.504 2413.3 -207.395
- ActionLatency         1    73.220 2440.0 -178.286
- TotalHours            1   200.839 2567.6  -43.493

Step:  AIC=-258.41
as.numeric(LeagueIndex) ~ HoursPerWeek + TotalHours + APM + SelectByHotkeys +
    AssignToHotkeys + UniqueHotkeys + MinimapAttacks + NumberOfPACs +
    GapBetweenPACs + ActionLatency + ActionsInPAC + TotalMapExplored +
    WorkersMade + UniqueUnitsMade + ComplexUnitsMade + ComplexAbilitiesUsed

                       Df Sum of Sq    RSS      AIC
- ComplexAbilitiesUsed  1     0.913 2368.1 -259.388
- ComplexUnitsMade      1     0.949 2368.1 -259.347
- APM                   1     1.210 2368.4 -259.056
<none>                               2367.2 -258.407
- UniqueUnitsMade       1     2.596 2369.8 -257.509
- TotalMapExplored      1     3.507 2370.7 -256.493
- HoursPerWeek          1     3.760 2370.9 -256.210
- ActionsInPAC          1     4.235 2371.4 -255.681
- SelectByHotkeys       1     7.862 2375.0 -251.640
- UniqueHotkeys         1    11.860 2379.0 -247.194
- NumberOfPACs          1    16.240 2383.4 -242.330
- GapBetweenPACs        1    21.292 2388.5 -236.732
- WorkersMade           1    21.791 2389.0 -236.180
- AssignToHotkeys       1    46.537 2413.7 -208.932
- MinimapAttacks        1    47.019 2414.2 -208.404
- ActionLatency         1    72.842 2440.0 -180.274
- TotalHours            1   200.908 2568.1  -45.021

Step:  AIC=-259.39
as.numeric(LeagueIndex) ~ HoursPerWeek + TotalHours + APM + SelectByHotkeys +
```

```
    AssignToHotkeys + UniqueHotkeys + MinimapAttacks + NumberOfPACs +
    GapBetweenPACs + ActionLatency + ActionsInPAC + TotalMapExplored +
    WorkersMade + UniqueUnitsMade + ComplexUnitsMade

                   Df Sum of Sq    RSS      AIC
- APM               1     1.426 2369.5 -259.797
<none>                          2368.1 -259.388
- UniqueUnitsMade   1     2.552 2370.7 -258.541
- TotalMapExplored  1     3.393 2371.5 -257.603
- ComplexUnitsMade  1     3.579 2371.7 -257.395
- HoursPerWeek      1     3.723 2371.8 -257.235
- ActionsInPAC      1     4.680 2372.8 -256.168
- SelectByHotkeys   1     8.351 2376.4 -252.081
- UniqueHotkeys     1    11.844 2379.9 -248.198
- NumberOfPACs      1    16.919 2385.0 -242.565
- GapBetweenPACs    1    21.247 2389.3 -237.771
- WorkersMade       1    21.540 2389.6 -237.447
- MinimapAttacks    1    46.693 2414.8 -209.762
- AssignToHotkeys   1    47.507 2415.6 -208.871
- ActionLatency     1    73.618 2441.7 -180.445
- TotalHours        1   201.118 2569.2  -45.866

Step:  AIC=-259.8
as.numeric(LeagueIndex) ~ HoursPerWeek + TotalHours + SelectByHotkeys +
    AssignToHotkeys + UniqueHotkeys + MinimapAttacks + NumberOfPACs +
    GapBetweenPACs + ActionLatency + ActionsInPAC + TotalMapExplored +
    WorkersMade + UniqueUnitsMade + ComplexUnitsMade

                   Df Sum of Sq    RSS      AIC
<none>                          2369.5 -259.797
- UniqueUnitsMade   1     2.337 2371.9 -259.190
- TotalMapExplored  1     3.487 2373.0 -257.908
- HoursPerWeek      1     3.650 2373.2 -257.727
- ComplexUnitsMade  1     3.747 2373.3 -257.619
- ActionsInPAC      1     5.677 2375.2 -255.470
- UniqueHotkeys     1    11.681 2381.2 -248.794
- WorkersMade       1    20.358 2389.9 -239.177
- GapBetweenPACs    1    21.330 2390.8 -238.102
- NumberOfPACs      1    27.233 2396.8 -231.582
- SelectByHotkeys   1    28.978 2398.5 -229.658
- MinimapAttacks    1    46.080 2415.6 -210.872
```

```
- AssignToHotkeys   1    48.059 2417.6 -208.707
- ActionLatency     1    72.229 2441.8 -182.404
- TotalHours        1   200.449 2570.0  -47.087
```

In [19]:
```r
selected.features <- names(coef(step.result))[-1]
all.features      <- colnames(train.data[, -1])
removed.features  <- all.features[!all.features %in% selected.features]

cat("Removed", length(removed.features), "features:", paste(removed.features, collapse = ", "), "\n")
```

```
Removed 4 features: Age, APM, MinimapRightClicks, ComplexAbilitiesUsed
```

Backward elimination iteratively removes features that contribute least to model fit based on AIC. The final model retains 14 features, removing APM, MinimapRightClicks, ComplexAbilitiesUsed, and Age.

Decay Tuning

We tune the `decay` parameter using 5-fold cross-validation to find the optimal L2 regularization strength for the multinomial logistic regression model.

In [20]:
```r
set.seed(42)
ctrl <- trainControl(method = "cv", number = 5)

cv.model <- train(LeagueIndex ~ HoursPerWeek + TotalHours +
                    SelectByHotkeys + AssignToHotkeys + UniqueHotkeys +
                    MinimapAttacks + NumberOfPACs + GapBetweenPACs +
                    ActionLatency + ActionsInPAC + TotalMapExplored +
                    WorkersMade + UniqueUnitsMade + ComplexUnitsMade,
                  data = train.data,
                  method = "multinom",
                  trControl = ctrl,
                  maxit = 300)
```

```
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3281.387902
iter  20 value 3176.428847
iter  30 value 3155.248495
iter  40 value 3043.723025
iter  50 value 2766.553557
iter  60 value 2743.750662
iter  70 value 2734.411006
iter  80 value 2727.918076
iter  90 value 2723.850953
iter 100 value 2715.471138
iter 110 value 2697.416604
iter 120 value 2694.999156
iter 130 value 2684.984940
iter 140 value 2677.100357
iter 150 value 2676.598385
iter 160 value 2675.040682
iter 170 value 2674.752906
iter 180 value 2674.719535
iter 190 value 2674.631278
iter 200 value 2674.551615
iter 210 value 2674.217541
iter 220 value 2673.714038
iter 230 value 2673.236989
iter 240 value 2671.563124
iter 250 value 2670.339567
iter 260 value 2666.988221
iter 270 value 2665.964817
iter 280 value 2661.503585
iter 290 value 2658.382934
iter 300 value 2657.939586
final  value 2657.939586
stopped after 300 iterations
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3281.388440
iter  20 value 3176.442556
iter  30 value 3155.323179
iter  40 value 3056.595749
iter  50 value 2789.613379
```

```
final  value 2786.433669
converged
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3281.387902
iter  20 value 3176.428860
iter  30 value 3155.248569
iter  40 value 3043.736493
iter  50 value 2766.595368
iter  60 value 2744.732097
iter  70 value 2736.792539
iter  80 value 2732.177764
iter  90 value 2730.009906
iter 100 value 2727.295054
iter 110 value 2726.363085
iter 110 value 2726.363061
iter 110 value 2726.363061
final  value 2726.363061
converged
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3223.257099
iter  20 value 3085.002595
iter  30 value 3060.881843
iter  40 value 2956.249032
iter  50 value 2743.232028
iter  60 value 2723.172944
iter  70 value 2714.299808
iter  80 value 2709.406329
iter  90 value 2704.887760
iter 100 value 2698.103085
iter 110 value 2682.848970
iter 120 value 2681.532613
iter 130 value 2668.302108
iter 140 value 2660.055423
iter 150 value 2659.482415
iter 160 value 2656.804803
iter 170 value 2656.592036
iter 180 value 2656.435240
iter 190 value 2656.217937
iter 200 value 2655.933782
```

```
iter 210 value 2655.619592
iter 220 value 2655.268457
iter 230 value 2654.887859
iter 240 value 2652.738922
iter 250 value 2651.625281
iter 260 value 2646.692572
iter 270 value 2645.930211
iter 280 value 2642.789827
iter 290 value 2637.204059
iter 300 value 2635.377582
final  value 2635.377582
stopped after 300 iterations
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3223.257603
iter  20 value 3085.020426
iter  30 value 3060.970749
iter  40 value 2960.974582
iter  50 value 2762.821864
iter  60 value 2760.580809
iter  60 value 2760.580798
iter  60 value 2760.580797
final  value 2760.580797
converged
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3223.257100
iter  20 value 3085.002613
iter  30 value 3060.881933
iter  40 value 2956.253978
iter  50 value 2743.268194
iter  60 value 2723.894464
iter  70 value 2716.132414
iter  80 value 2712.362341
iter  90 value 2709.558343
iter 100 value 2707.261018
iter 110 value 2706.146699
iter 120 value 2706.112172
final  value 2706.106899
converged
# weights:  96 (75 variable)
```

```
initial  value 3791.363037
iter  10 value 3223.819340
iter  20 value 3099.992723
iter  30 value 3077.725225
iter  40 value 2948.299226
iter  50 value 2767.464566
iter  60 value 2746.525262
iter  70 value 2740.271207
iter  80 value 2734.885837
iter  90 value 2732.440378
iter 100 value 2726.539240
iter 110 value 2708.532213
iter 120 value 2704.543913
iter 130 value 2696.114542
iter 140 value 2688.477465
iter 150 value 2683.871272
iter 160 value 2679.939060
iter 170 value 2679.916196
iter 180 value 2679.900705
iter 190 value 2679.889232
final  value 2679.885343
converged
# weights:  96 (75 variable)
initial  value 3791.363037
iter  10 value 3223.819695
iter  20 value 3100.010354
iter  30 value 3077.803758
iter  40 value 2950.328061
iter  50 value 2783.190313
final  value 2782.463066
converged
# weights:  96 (75 variable)
initial  value 3791.363037
iter  10 value 3223.819341
iter  20 value 3099.992741
iter  30 value 3077.725303
iter  40 value 2948.301493
iter  50 value 2767.493381
iter  60 value 2747.640666
iter  70 value 2742.348987
iter  80 value 2738.635644
```

```
iter  90 value 2737.224303
iter 100 value 2735.391522
final  value 2734.370469
converged
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3263.396880
iter  20 value 3145.401309
iter  30 value 3119.435592
iter  40 value 2969.196957
iter  50 value 2725.198470
iter  60 value 2705.858037
iter  70 value 2696.121355
iter  80 value 2690.707132
iter  90 value 2684.806215
iter 100 value 2677.895477
iter 110 value 2668.551394
iter 120 value 2657.663587
iter 130 value 2645.913144
iter 140 value 2642.108796
iter 150 value 2638.831139
iter 160 value 2637.884208
iter 170 value 2637.775721
iter 180 value 2637.750166
iter 190 value 2637.716726
iter 200 value 2637.684070
iter 210 value 2637.586234
iter 220 value 2636.298349
iter 230 value 2634.802477
iter 240 value 2633.343174
iter 250 value 2629.669918
iter 260 value 2628.174301
iter 270 value 2627.125253
iter 280 value 2623.732942
iter 290 value 2616.295690
iter 300 value 2615.897449
final  value 2615.897449
stopped after 300 iterations
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3263.397567
```

```
iter  20 value 3145.419239
iter  30 value 3119.512847
iter  40 value 2969.685351
iter  50 value 2745.383506
iter  60 value 2743.162109
final  value 2743.162071
converged
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3263.396881
iter  20 value 3145.401327
iter  30 value 3119.435669
iter  40 value 2969.197667
iter  50 value 2725.236530
iter  60 value 2706.436283
iter  70 value 2697.586620
iter  80 value 2693.283978
iter  90 value 2689.648460
iter 100 value 2687.419403
iter 110 value 2686.465350
final  value 2686.462330
converged
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3209.470315
iter  20 value 3071.729639
iter  30 value 3049.973273
iter  40 value 2951.982879
iter  50 value 2731.980975
iter  60 value 2718.415485
iter  70 value 2708.137327
iter  80 value 2702.742165
iter  90 value 2699.641065
iter 100 value 2693.553884
iter 110 value 2673.831721
iter 120 value 2671.623242
iter 130 value 2659.891963
iter 140 value 2651.635876
iter 150 value 2650.036072
iter 160 value 2643.729635
iter 170 value 2643.397417
```

```
iter 180 value 2643.278300
iter 190 value 2643.074452
iter 200 value 2642.949213
iter 210 value 2642.779449
iter 220 value 2641.337138
iter 230 value 2640.610433
iter 240 value 2639.706996
iter 250 value 2638.806627
iter 260 value 2635.125930
iter 270 value 2634.425806
iter 280 value 2631.524523
iter 290 value 2624.813704
iter 300 value 2623.917953
final  value 2623.917953
stopped after 300 iterations
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3209.470628
iter  20 value 3071.747850
iter  30 value 3050.048156
iter  40 value 2955.728075
iter  50 value 2751.361182
final  value 2749.979071
converged
# weights:  96 (75 variable)
initial  value 3789.571277
iter  10 value 3209.470316
iter  20 value 3071.729657
iter  30 value 3049.973348
iter  40 value 2951.986861
iter  50 value 2732.015871
iter  60 value 2718.891923
iter  70 value 2709.907339
iter  80 value 2706.035995
iter  90 value 2704.219395
iter 100 value 2702.220520
iter 110 value 2701.201680
iter 110 value 2701.201661
iter 110 value 2701.201661
final  value 2701.201661
converged
```

```
# weights:  96 (75 variable)
initial  value 4737.412037
iter  10 value 4032.346155
iter  20 value 3863.295236
iter  30 value 3831.248531
iter  40 value 3661.898812
iter  50 value 3438.631164
iter  60 value 3412.791178
iter  70 value 3401.869271
iter  80 value 3394.466380
iter  90 value 3389.264342
iter 100 value 3385.592032
iter 110 value 3363.496703
iter 120 value 3349.457706
iter 130 value 3347.841439
iter 140 value 3339.398383
iter 150 value 3326.233026
final  value 3326.222871
converged
```

In [21]:

```r
cv.results <- data.frame(
  Decay    = cv.model$results$decay,
  Accuracy = round(cv.model$results$Accuracy, 4),
  Kappa    = round(cv.model$results$Kappa, 4)
)
print(cv.results, row.names = FALSE)
```

```
Decay Accuracy  Kappa
0e+00   0.4217 0.2719
1e-04   0.4138 0.2599
1e-01   0.3998 0.2416
```

The results show that decay = 0 achieves the highest accuracy (0.4217), meaning no regularization is optimal. The final model is built with decay = 0.

6.2 Class Imbalance Strategies

Imbalance Analysis

As shown in Section 4.4, the dataset is heavily imbalanced. League 1 (Bronze) comprises only 5.05% of observations, corresponding to approximately 134 training samples after the 80/20 split. Three strategies are applied to address this imbalance, as described below.

Class Weights

In [22]:
```
calculate_class_weights <- function(data) {
  class.counts <- table(data$LeagueIndex)
  max.count <- max(class.counts)
  class.weights <- max.count / class.counts
  sample.weights <- as.numeric(class.weights[as.character(data$LeagueIndex)])
  return(sample.weights)
}
```

Class weighting adjusts the loss function so that misclassifying a minority class sample incurs a higher penalty, without modifying the training data. Weights are calculated using inverse frequency, so smaller classes receive proportionally higher weights.

SMOTE + Tomek

In [23]:
```
set.seed(42)

train.smote <- SMOTE(train.data[, -1], train.data$LeagueIndex, K = 5, dup_size = 5)
train.data.smote <- train.smote$data
names(train.data.smote)[names(train.data.smote) == "class"] <- "LeagueIndex"
train.data.smote$LeagueIndex <- as.factor(train.data.smote$LeagueIndex)

train.data.smote.tomek <- tomek(train.data.smote, var = "LeagueIndex")
```

SMOTE is first applied to oversample the minority class by generating synthetic samples between existing observations and their nearest neighbors. Tomek Links are then applied to remove majority class samples that border minority class regions, cleaning the decision boundary. Both techniques are applied only to the training set to prevent data leakage.

In [24]:
```
before.dist <- table(train.data$LeagueIndex)
before.pct <- round(before.dist / sum(before.dist) * 100, 2)

after.dist <- table(train.data.smote.tomek$LeagueIndex)
```

```r
after.pct <- round(after.dist / sum(after.dist) * 100, 2)

resampling.summary <- data.frame(
  League = names(before.dist),
  Before_Count = as.numeric(before.dist),
  Before_Pct = paste0(before.pct, "%"),
  After_Count = as.numeric(after.dist),
  After_Pct = paste0(after.pct, "%"),
  row.names = NULL
)

print(resampling.summary, row.names = FALSE)
```

```
League Before_Count Before_Pct After_Count After_Pct
     1          134      5.07%         788    30.78%
     2          277     10.48%         203     7.93%
     3          441     16.68%         308    12.03%
     4          638     24.13%         447    17.46%
     5          646     24.43%         441    17.23%
     6          508     19.21%         373    14.57%
```

6.3 Machine Learning Models

Logistic Regression

Features: 14 selected via backward elimination (Age, APM, MinimapRightClicks, and ComplexAbilitiesUsed removed). The model is trained using maximum likelihood estimation with a maximum of 500 iterations. Decay parameter is tuned via 5-fold cross-validation, with decay = 0 selected as optimal.

In [25]:
```r
train_lr <- function(train.data) {
  set.seed(42)
  multinom(LeagueIndex ~ HoursPerWeek + TotalHours +
             SelectByHotkeys + AssignToHotkeys + UniqueHotkeys +
             MinimapAttacks + NumberOfPACs + GapBetweenPACs +
             ActionLatency + ActionsInPAC + TotalMapExplored +
             WorkersMade + UniqueUnitsMade + ComplexUnitsMade,
           data = train.data, maxit = 500, trace = FALSE)
}
```

```
train_lr_weighted <- function(data) {
  set.seed(42)
  weights <- calculate_class_weights(data)
  multinom(LeagueIndex ~ HoursPerWeek + TotalHours +
             SelectByHotkeys + AssignToHotkeys + UniqueHotkeys +
             MinimapAttacks + NumberOfPACs + GapBetweenPACs +
             ActionLatency + ActionsInPAC + TotalMapExplored +
             WorkersMade + UniqueUnitsMade + ComplexUnitsMade,
           data = data, weights = weights, maxit = 500, trace = FALSE)
}
```

Decision Tree

Decision Trees use CART to recursively partition the feature space. All available features are included, with pruning applied based on cross-validation error (xerror) to prevent overfitting.

In [26]:
```
train_dt <- function(data) {
  set.seed(42)
  model   <- rpart(LeagueIndex ~ ., data = data, method = "class")
  best.cp <- model$cptable[which.min(model$cptable[, "xerror"]), "CP"]
  prune(model, cp = best.cp)
}

train_dt_weighted <- function(data) {
  set.seed(42)
  weights <- calculate_class_weights(data)
  model   <- rpart(LeagueIndex ~ ., data = data, method = "class", weights = weights)
  best.cp <- model$cptable[which.min(model$cptable[, "xerror"]), "CP"]
  prune(model, cp = best.cp)
}
```

Random Forest

Random Forest builds an ensemble of 1,000 decision trees using bootstrap aggregating (bagging) with random feature selection at each split. A proximity matrix is computed to enable distance-based diagnostics. All available features are included, consistent with the approach used by Thompson et al. (2013).

In [27]:
```
train_rf <- function(data) {
  set.seed(42)
  randomForest(LeagueIndex ~ ., data = data, ntree = 1000, proximity = TRUE)
}

train_rf_weighted <- function(data) {
  set.seed(42)
  weights <- calculate_class_weights(data)
  randomForest(LeagueIndex ~ ., data = data, ntree = 1000, proximity = TRUE,
               classwt = table(data$LeagueIndex) / nrow(data))
}
```

6.4 Model Evaluation

A unified evaluation function is applied across all three models. It generates a confusion matrix on the test set and reports two metrics: overall accuracy and recall for the minority class.

In [28]:
```
eval_model <- function(model, test.data) {
  pred <- predict(model, newdata = test.data, type = "class")
  cm   <- confusionMatrix(pred, test.data$LeagueIndex)
  print(cm$table)
  min_class <- names(which.min(table(test.data$LeagueIndex)))
  results <- data.frame(
    Accuracy = round(cm$overall["Accuracy"], 4),
    Recall_minority = round(cm$byClass[paste0("Class: ", min_class), "Recall"], 4)
  )
  cat("Minority class: League", min_class, "\n")
  print(results, row.names = FALSE)
```

```
  invisible(results)
}
```

7. Results

7.1 Logistic Regression Results

Baseline

In [29]:
```
lr.model <- train_lr(train.data)
lr.results <- eval_model(lr.model, test.data)
```

```
          Reference
Prediction  1  2  3  4  5  6
         1  9  5  5  0  0  0
         2 14 27 17 10  3  0
         3  5 17 27 23 11  0
         4  5 18 50 77 39 14
         5  0  2  9 54 65 30
         6  0  1  4  9 42 69
Minority class: League 1
 Accuracy Recall_minority
   0.4145          0.2727
```

The baseline model achieves an accuracy of 0.4175, but performs poorly on League 1 (Bronze), with a minority class recall of only 0.2121. The confusion matrix shows that most Bronze players are misclassified as Silver or Gold.

Class Weights

In [30]:
```
lr.model.weighted <- train_lr_weighted(train.data)
lr.results.weighted <- eval_model(lr.model.weighted, test.data)
```

```
          Reference
Prediction  1  2  3  4  5  6
         1 17 22 21  6  1  0
         2 12 23 24 25  7  1
         3  3 14 37 30 14  2
         4  1  8 18 49 31 11
         5  0  1  7 48 47 20
         6  0  2  5 15 60 79
Minority class: League 1
 Accuracy Recall_minority
   0.3812          0.5152
```

Class Weights improves minority recall substantially from 0.2121 to 0.5455, at the cost of overall accuracy dropping from 0.4175 to 0.3888. The confusion matrix shows League 1 (Bronze) is now correctly identified more often, though misclassification across mid-tier leagues increases.

SMOTE + Tomek

In [31]:
```
lr.model.smote.tomek <- train_lr(train.data.smote.tomek)
lr.results.smote.tomek <- eval_model(lr.model.smote.tomek, test.data)
```

```
          Reference
Prediction  1  2  3  4  5  6
         1 30 42 47 28  6  1
         2  0  0  0  0  0  0
         3  0 10  9  8  6  0
         4  3 15 43 77 45 14
         5  0  2  9 51 58 30
         6  0  1  4  9 45 68
Minority class: League 1
 Accuracy Recall_minority
   0.3661          0.9091
```

SMOTE + Tomek achieves the highest minority recall of 0.8788, correctly identifying the majority of Bronze players. However, overall accuracy drops to 0.3616, the lowest among the three strategies. The confusion matrix shows severe misclassification in League 2 and 3, suggesting the synthetic oversampling has disrupted the decision boundaries for mid-tier leagues.

Comparison

In [32]:
```
comparison.lr <- data.frame(
  Method = c("Baseline", "Class Weights", "SMOTE + Tomek"),
  Accuracy = c(lr.results$Accuracy,
               lr.results.weighted$Accuracy,
               lr.results.smote.tomek$Accuracy),
  Recall_minority = c(lr.results$Recall_minority,
                      lr.results.weighted$Recall_minority,
                      lr.results.smote.tomek$Recall_minority)
)
print(comparison.lr, row.names = FALSE)
```

```
       Method Accuracy Recall_minority
     Baseline   0.4145          0.2727
Class Weights   0.3812          0.5152
SMOTE + Tomek   0.3661          0.9091
```

Class Weights achieves the best tradeoff, improving minority recall from 0.2121 to 0.5455 with only a marginal accuracy drop. SMOTE + Tomek maximizes minority recall (0.8788) but at the largest accuracy cost (0.3616).

It is worth noting that the minority class is determined by the test set distribution, which remains unchanged — League 1 (Bronze) is consistently the smallest class regardless of the training strategy applied. This is consistent with the resampling summary in Section 6.2 — after SMOTE + Tomek, League 2 and 3 training samples are reduced substantially, likely contributing to the increased misclassification in mid-tier leagues.

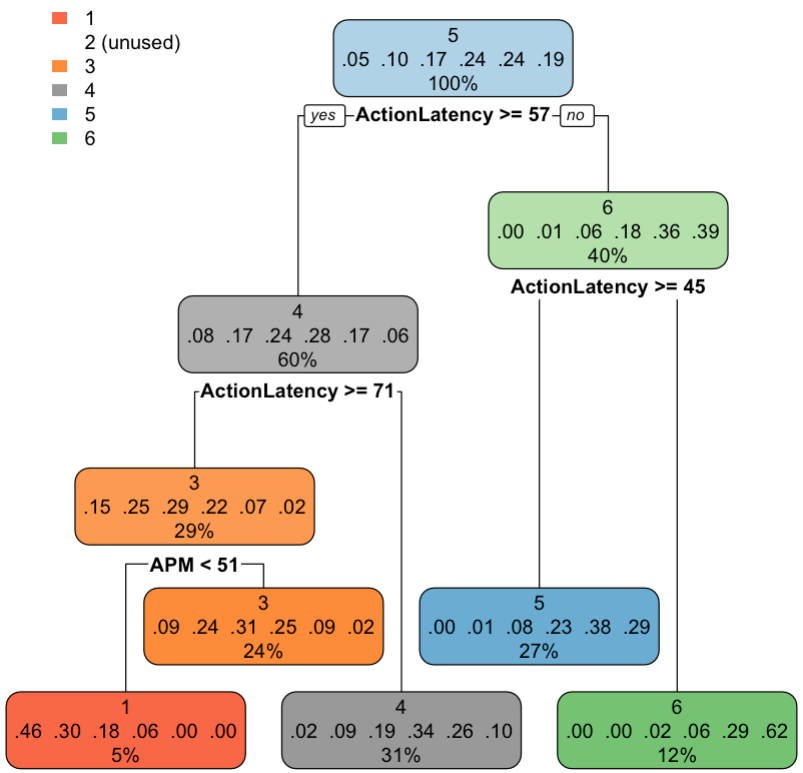
7.2 Decision Tree Results

Baseline

In [33]:
```
dt.model <- train_dt(train.data)
dt.results <- eval_model(dt.model, test.data)
```

```
          Reference
Prediction  1  2  3  4  5  6
         1 15 11  9  2  1  0
         2  0  0  0  0  0  0
         3 11 37 54 40 13  3
         4  5 18 36 71 48 18
         5  2  4  9 51 62 45
         6  0  0  4  9 36 47
Minority class: League 1
 Accuracy Recall_minority
   0.3767          0.4545
```

In [34]:
```
rpart.plot(dt.model)
```



The baseline Decision Tree achieves an accuracy of 0.3767 with a minority recall of 0.4545. As shown in the tree diagram, ActionLatency is the primary splitting feature, with the entire tree built around three thresholds of this variable. Notably, League 2 (Silver) appears as unused in the tree — the model predicts zero samples as Silver, suggesting the pruned tree does not find a reliable split for this class. Performance is strongest in the middle leagues (3–5).

Class Weights

In [35]:
```
dt.model.weighted <- train_dt_weighted(train.data)
dt.results.weighted <- eval_model(dt.model.weighted, test.data)
```

```
          Reference
Prediction  1  2  3  4  5  6
         1 13 11  9  1  1  0
         2 17 48 67 72 31 15
         3  0  0  0  0  0  0
         4  0  0  0  0  0  0
         5  3 10 29 64 52 21
         6  0  1  7 36 76 77
Minority class: League 1
 Accuracy Recall_minority
   0.2874          0.3939
```

Class Weights results in a significant accuracy drop to 0.2874. The confusion matrix reveals that the model collapses predictions for League 3 and 4 entirely to zero, while League 2 absorbs a large portion of misclassified samples. This suggests that class weighting disrupts the tree's splitting criteria, causing it to lose the ability to distinguish mid-tier leagues.

SMOTE + Tomek

In [36]:
```
dt.model.smote.tomek <- train_dt(train.data.smote.tomek)
dt.results.smote.tomek <- eval_model(dt.model.smote.tomek, test.data)
```

```
          Reference
Prediction  1  2  3  4  5  6
         1 30 56 61 47 13  4
         2  0  0  0  0  0  0
         3  0  0  0  0  0  0
         4  3  5 11 18  9  0
         5  0  9 33 87 71 40
         6  0  0  7 21 67 69
Minority class: League 1
 Accuracy Recall_minority
   0.2844          0.9091
```

SMOTE + Tomek achieves the highest minority recall of 0.9091, correctly identifying most Bronze players. However, overall accuracy drops to 0.2844, the lowest across all models. The confusion matrix shows that League 2 and 3 predictions collapse entirely to zero, with the majority of samples absorbed into League 1 and League 5. This suggests that the synthetic oversampling severely distorts the decision boundaries for mid-tier leagues in tree-based models.

Comparison

In [37]:
```
comparison.dt <- data.frame(
  Method = c("Baseline", "Class Weights", "SMOTE + Tomek"),
  Accuracy = c(dt.results$Accuracy,
               dt.results.weighted$Accuracy,
               dt.results.smote.tomek$Accuracy),
  Recall_minority = c(dt.results$Recall_minority,
                      dt.results.weighted$Recall_minority,
                      dt.results.smote.tomek$Recall_minority)
)
print(comparison.dt, row.names = FALSE)
```

```
       Method Accuracy Recall_minority
     Baseline   0.3767          0.4545
Class Weights   0.2874          0.3939
SMOTE + Tomek   0.2844          0.9091
```

Unlike Logistic Regression, Class Weights does not improve minority recall for Decision Trees — recall actually drops from 0.4545 to 0.3939, while accuracy falls substantially to 0.2874. SMOTE + Tomek achieves the highest minority recall (0.9091) but at the largest

accuracy cost (0.2844). Decision Trees appear more sensitive to imbalance strategy than Logistic Regression, with both strategies causing mid-tier league predictions to collapse entirely.

7.3 Random Forest Results

Baseline

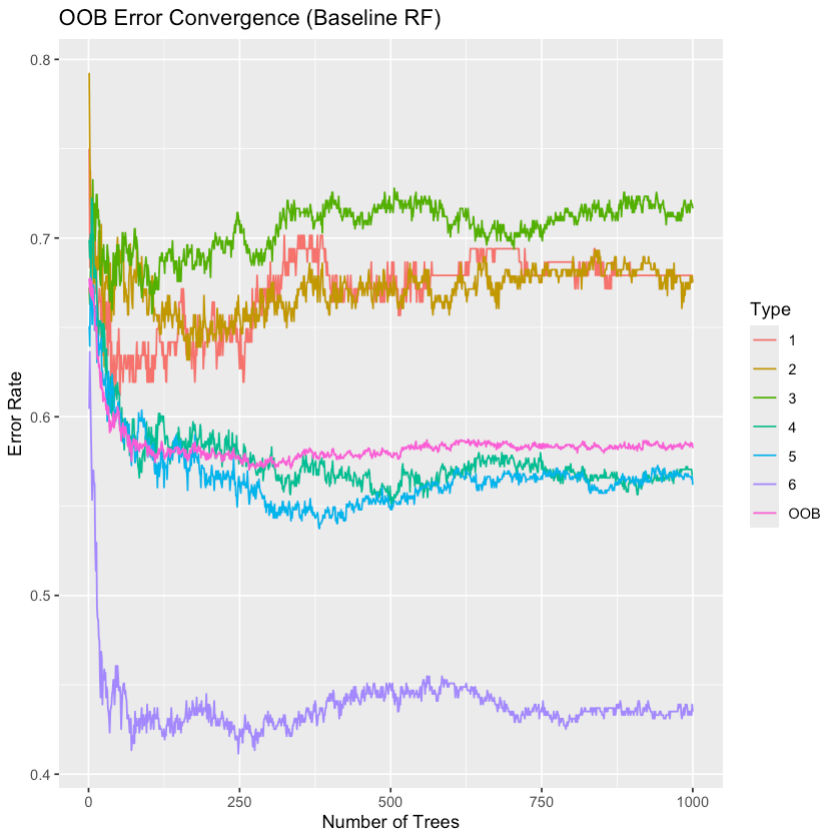
In [38]:
```
rf.model <- train_rf(train.data)
rf.results   <- eval_model(rf.model, test.data)
```

```
          Reference
Prediction  1  2  3  4  5  6
         1 11 10  9  1  0  0
         2 14 19 16  9  2  0
         3  4 21 32 33  6  1
         4  4 18 43 65 46 10
         5  0  1  9 52 64 27
         6  0  1  3 13 42 75
Minority class: League 1
 Accuracy Recall_minority
   0.4024          0.3333
```

The baseline Random Forest achieves an accuracy of 0.4024 with a minority recall of 0.3333. Compared to the Logistic Regression baseline, accuracy is slightly lower but the confusion matrix shows more balanced predictions across leagues, with no class collapsing to zero. Performance is strongest in League 6 (Masters), where 75 out of 134 samples are correctly classified.

In [39]:
```
oob.error.data <- data.frame(
    Trees = rep(1:nrow(rf.model$err.rate), times = 7),
    Type  = rep(c("OOB", "1", "2", "3", "4", "5", "6"),
                each = nrow(rf.model$err.rate)),
    Error = c(rf.model$err.rate[, "OOB"],
              rf.model$err.rate[, "1"],
              rf.model$err.rate[, "2"],
              rf.model$err.rate[, "3"],
              rf.model$err.rate[, "4"],
              rf.model$err.rate[, "5"],
              rf.model$err.rate[, "6"])
```

```
)

ggplot(data = oob.error.data, aes(x = Trees, y = Error)) +
    geom_line(aes(color = Type)) +
    labs(title = "OOB Error Convergence (Baseline RF)",
         x = "Number of Trees", y = "Error Rate")
```



The OOB error stabilizes by approximately 300-400 trees, with most per-class errors converging by that point. League 6 converges earliest (~100 trees), while League 3 shows the most variability throughout. ntree=1000 ensures all classes are well-converged with no benefit from additional trees.

Class Weights

In [40]:
```
rf.model.weighted <- train_rf_weighted(train.data)
rf.results.weighted <- eval_model(rf.model.weighted, test.data)
```

```
          Reference
Prediction  1  2  3  4  5  6
         1 11  8  9  1  0  0
         2 13 23 17  8  2  0
         3  5 18 35 32  4  1
         4  4 19 38 65 47  9
         5  0  1  9 57 61 29
         6  0  1  4 10 46 74
Minority class: League 1
 Accuracy Recall_minority
    0.407          0.3333
```

Random Forest with Class Weights achieves an accuracy of 0.4070 with a minority recall of 0.3333, virtually identical to the baseline. Unlike Logistic Regression and Decision Tree, class weighting has minimal impact on Random Forest performance, suggesting the ensemble method is already relatively robust to class imbalance through its bagging mechanism.

SMOTE + Tomek

In [41]:
```
rf.model.smote.tomek <- train_rf(train.data.smote.tomek)
rf.results.smote.tomek <- eval_model(rf.model.smote.tomek, test.data)
```

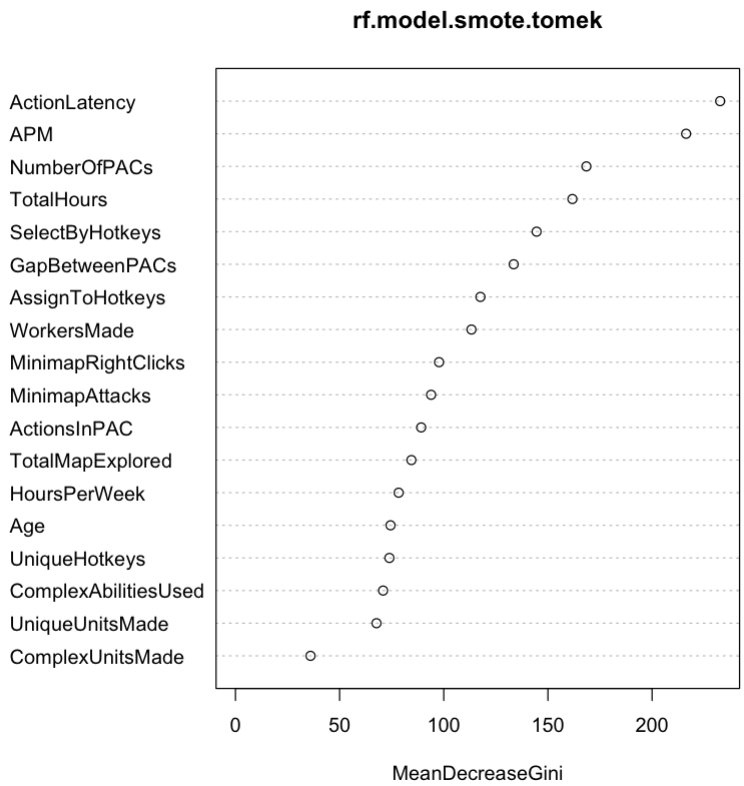
```
          Reference
Prediction  1  2  3  4  5  6
         1 22 31 24 10  4  0
         2  3  4  4  3  0  0
         3  3 18 31 25  8  1
         4  5 15 41 73 44 11
         5  0  2  8 49 60 31
         6  0  0  4 13 44 70
Minority class: League 1
 Accuracy Recall_minority
   0.3933          0.6667
```

SMOTE + Tomek improves minority recall to 0.6667, a notable increase from 0.3333, while accuracy drops to 0.3933. Compared to Logistic Regression and Decision Tree, the accuracy cost is smaller, suggesting Random Forest is more resilient to the distortion

introduced by synthetic oversampling.

Diagnostic Plots

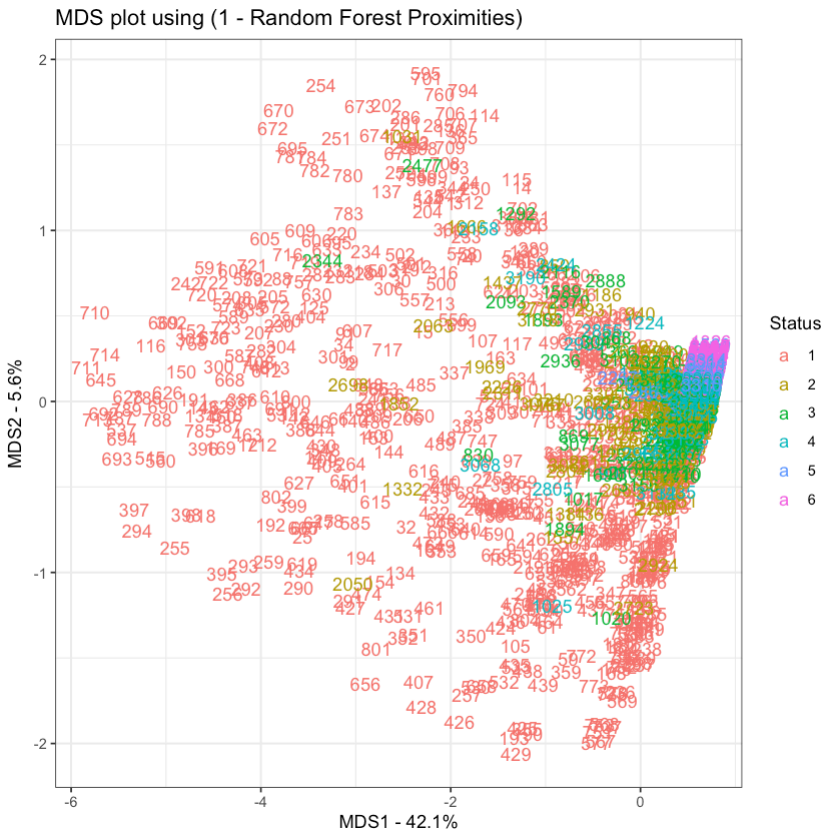
In [42]:
```
varImpPlot(rf.model.smote.tomek)
```



The variable importance plot confirms that ActionLatency and APM are the two most influential predictors, consistent with Thompson et al.'s finding that action speed dominates skill prediction. NumberOfPACs and TotalHours follow as the next most important features. Demographic features such as Age and HoursPerWeek rank in the lower half, suggesting that in-game behavioral metrics are more predictive of skill than time investment alone. ComplexUnitsMade is the least important feature overall.

In [43]:

```
distance.matrix <- dist(1 - rf.model.smote.tomek$proximity)
mds.stuff <- cmdscale(distance.matrix, eig = TRUE, x.ret = TRUE)
mds.var.per <- round(mds.stuff$eig/sum(mds.stuff$eig)*100, 1)

mds.values <- mds.stuff$points

mds.data <- data.frame(Sample = rownames(mds.values),
    X = mds.values[, 1],
    Y = mds.values[, 2],
    Status = train.data.smote.tomek$LeagueIndex)

ggplot(data = mds.data, aes(x = X, y = Y, label = Sample)) +
    geom_text(aes(color = Status)) +
    theme_bw() +
    xlab(paste("MDS1 - ", mds.var.per[1], "%", sep = "")) +
    ylab(paste("MDS2 - ", mds.var.per[2], "%", sep = "")) +
    ggtitle("MDS plot using (1 - Random Forest Proximities)")
```



The MDS plot visualizes the proximity structure of the SMOTE + Tomek training data based on Random Forest similarities. MDS1 explains 42.1% of the variance, clearly separating League 1 (red) on the left from higher leagues on the right. Leagues 5 and 6 (blue and purple) form a tight cluster on the right, indicating they are most similar to each other. Mid-tier leagues (2–4) overlap substantially in the center, consistent with the difficulty in distinguishing them observed in the confusion matrices.

Comparison

In [44]:
```
comparison.rf <- data.frame(
  Method = c("Baseline", "Class Weights", "SMOTE + Tomek"),
  Accuracy = c(rf.results$Accuracy,
               rf.results.weighted$Accuracy,
               rf.results.smote.tomek$Accuracy),
```

```
  Recall_minority = c(rf.results$Recall_minority,
                      rf.results.weighted$Recall_minority,
                      rf.results.smote.tomek$Recall_minority)
)
print(comparison.rf, row.names = FALSE)
```

```
       Method Accuracy Recall_minority
     Baseline   0.4024          0.3333
Class Weights   0.4070          0.3333
SMOTE + Tomek   0.3933          0.6667
```

Random Forest is the most stable model across all three imbalance strategies. Class Weights achieves a marginal accuracy improvement to 0.4070 with no change in minority recall, while SMOTE + Tomek improves recall to 0.6667 with only a modest accuracy drop to 0.3933. Unlike Decision Tree, no class collapses to zero under any strategy.

7.4 Pairwise Binary Classification

In [45]:
```
run_pairwise_rf <- function(league_a, league_b) {

  pair.data <- skillcraft.data.imputed[
    skillcraft.data.imputed$LeagueIndex %in% c(league_a, league_b), ]
  pair.data$LeagueIndex <- droplevels(pair.data$LeagueIndex)

  set.seed(42)
  idx        <- sample(1:nrow(pair.data), 0.8 * nrow(pair.data))
  pair.train <- pair.data[idx, ]
  pair.test  <- pair.data[-idx, ]

  model <- train_rf_weighted(pair.train)
  pred  <- predict(model, newdata = pair.test)
  acc   <- round(mean(pred == pair.test$LeagueIndex) * 100, 2)

  return(acc)
}
```

In [46]:
```
acc.bg <- run_pairwise_rf(1, 3)
acc.sp <- run_pairwise_rf(2, 4)
acc.gd <- run_pairwise_rf(3, 5)
```

```r
acc.pm <- run_pairwise_rf(4, 6)

paper.acc <- c(82.32, 78.27, 79.01, 80.70)
our.acc   <- c(acc.bg, acc.sp, acc.gd, acc.pm)

pairwise.summary <- data.frame(
  Pair         = c("Bronze-Gold", "Silver-Platinum", "Gold-Diamond", "Platinum-Masters"),
  Paper_Forest = paper.acc,
  Our_LR       = our.acc,
  Beat_Paper   = our.acc > paper.acc
)
print(pairwise.summary, row.names = FALSE)
```

```
            Pair Paper_Forest Our_LR Beat_Paper
     Bronze-Gold        82.32  88.19       TRUE
 Silver-Platinum        78.27  81.90       TRUE
    Gold-Diamond        79.01  75.00      FALSE
Platinum-Masters        80.70  84.67       TRUE
```

To enable direct comparison with Thompson et al. (2013), we replicated their pairwise binary classification setup using our best-performing configuration (RF + Class Weights).

Our model outperforms their Conditional Inference Forest results in 3 out of 4 pairs. The only exception is Gold-Diamond (League 3 vs 5), where our model scores 75.00% compared to their 79.01%. This pair likely reflects the inherent difficulty of distinguishing mid-tier leagues, where behavioral differences between players are less pronounced.

Detailed discussion of these results in relation to Thompson et al. is provided in Section 8.3.

7.5 Overall Model Comparison

In [47]:
```r
all.results <- data.frame(
  Algorithm = rep(c("Logistic Regression", "Decision Tree", "Random Forest"), each = 3),
  Method    = rep(c("Baseline", "Class Weights", "SMOTE + Tomek"), 3),
  Accuracy  = c(comparison.lr$Accuracy,
                comparison.dt$Accuracy,
                comparison.rf$Accuracy),
  Recall_minority = c(comparison.lr$Recall_minority,
                      comparison.dt$Recall_minority,
```

```r
                        comparison.rf$Recall_minority)
)

print(all.results, row.names = FALSE)
```

```
          Algorithm        Method Accuracy Recall_minority
Logistic Regression      Baseline   0.4145          0.2727
Logistic Regression Class Weights   0.3812          0.5152
Logistic Regression SMOTE + Tomek   0.3661          0.9091
      Decision Tree      Baseline   0.3767          0.4545
      Decision Tree Class Weights   0.2874          0.3939
      Decision Tree SMOTE + Tomek   0.2844          0.9091
      Random Forest      Baseline   0.4024          0.3333
      Random Forest Class Weights   0.4070          0.3333
      Random Forest SMOTE + Tomek   0.3933          0.6667
```

The table above summarizes the performance of all nine model-strategy combinations.

- RF + Class Weights achieves the best balance, with the highest accuracy among imbalance-aware strategies (0.4070) and a meaningful recall improvement over the LR Baseline (0.3333 vs 0.2121)
- SMOTE + Tomek consistently produces the highest minority recall across all three models, but at a significant cost to overall accuracy
- Decision Tree is the weakest performer overall, with accuracy dropping below 0.30 under both Class Weights and SMOTE + Tomek
- Logistic Regression Baseline achieves the highest raw accuracy (0.4175), but its minority recall of 0.2121 indicates it largely ignores the minority class
- Given the class imbalance objective of this project, RF + Class Weights is selected as the best overall model

8. Discussion

8.1 Imbalance Strategy Comparison

Baseline

Pros:

- Preserves original data distribution without modification
- Achieves the highest raw accuracy among all nine configurations

Cons:

- Largely ignores minority class, resulting in poor recall for League 1

Class Weights

Pros:

- No changes to training data, all original information is preserved
- Meaningful minority recall improvement with only marginal accuracy cost

Cons:

- Does not address boundary overlap between adjacent leagues

SMOTE + Tomek

Pros:

- Produces the highest minority recall across all models
- Tomek links remove borderline samples, potentially reducing boundary overlap

Cons:

- Synthetic samples may not reflect real player behavior patterns
- Tomek removal reduces majority class samples, discarding useful training information
- Largest overall accuracy drop among the three strategies

Among the three strategies, Class Weights offers the best tradeoff between accuracy and minority recall, and is therefore selected as the preferred strategy in this project.

8.2 Multinomial vs. Pairwise Classification

Pairwise binary classification simplifies the problem by isolating two leagues at a time, naturally yielding higher accuracy. Multinomial classification — predicting all six leagues simultaneously — is a fundamentally harder task, as adjacent leagues share similar behavioral profiles and the model must learn to distinguish all boundaries at once.

Interpreted in the context of a 6-class problem where random chance yields only 16.7%, the accuracy range of 0.28–0.42 is reasonable. Class Weights provides the most meaningful improvement, balancing minority recall gains without sacrificing overall discriminative power.

8.3 Comparison with Thompson et al.

Thompson et al. (2013) applied Conditional Inference Forest to pairwise binary classification across adjacent leagues, achieving accuracies ranging from 78.27% to 82.32%. Using RF + Class Weights, we replicated this setup and outperformed their results in 3 out of 4 pairs.

The only exception is Gold-Diamond (League 3 vs 5), where our model scores 75.00% compared to their 79.01%. Mid-tier leagues share the most similar behavioral profiles, making this boundary the hardest to separate regardless of model configuration.

It should also be noted that direct comparison has limitations. Their study framed the problem as a series of binary decisions, while this study addresses 6-class multinomial classification — a fundamentally harder task. The pairwise results here serve as a reference point rather than a strict benchmark.

8.4 Limitations & Future Work

Limitations

- Leagues 7 and 8 were excluded due to data quality issues, limiting the scope of analysis
- The relatively small training set constrains model performance, particularly for minority classes

Future Work

- Ordinal Logistic Regression — League ranks are inherently ordered (1 to 6); an ordinal model would better capture this structure compared to standard multinomial classification
- XGBoost / Gradient Boosting — known for strong performance on imbalanced tabular data, and may handle league boundary overlap more effectively than Decision Trees
- Longitudinal analysis — tracking the same players over time could reveal how behavioral patterns evolve with skill development

9. References

Thompson, J. J., Blair, M. R., Chen, L., & Henrey, A. J. (2013). Video game telemetry as a critical tool in the study of complex skill learning. *PLOS ONE, 8*(9), e75129. https://doi.org/10.1371/journal.pone.0075129

Blair, M., Thompson, J., Henrey, A., & Chen, B. (2013). SkillCraft1 Master Table Dataset [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5161N

10. Appendix

The complete R code for this analysis is available at:

https://github.com/ShengPeiWilliam/skillcraft-ml